

Try free

Have questions? [Contact us](#)

Already have an account? [Log in](#)

Elasticsearch Guide:

8.9 (current) v

- What is Elasticsearch? >
- What's new in 8.9
- Set up Elasticsearch >
- Upgrade Elasticsearch >
- Index modules >
- Mapping >
- Text analysis >
- Index templates >
- Data streams >
- Ingest pipelines >
- Aliases
- Search your data >
- Query DSL >
- Aggregations >
- Geospatial analysis
- EQL >
- SQL >
- Scripting >
- Data management >
- Autoscaling >
- Monitor a cluster >
- Roll up or transform your data >

Elastic Docs > Elasticsearch Guide [8.9] > REST APIs > Document APIs

?refresh



The [Index](#), [Update](#), [Delete](#), and [Bulk](#) APIs support setting `refresh` to control when changes made by this request are made visible to search. These are the allowed values:

Empty string or `true`

Refresh the relevant primary and replica shards (not the whole index) immediately after the operation occurs, so that the updated document appears in search results immediately. This should **ONLY** be done after careful thought and verification that it does not lead to poor performance, both from an indexing and a search standpoint.

`wait_for`

Wait for the changes made by the request to be made visible by a refresh before replying. This doesn't force an immediate refresh, rather, it waits for a refresh to happen. Elasticsearch automatically refreshes shards that have changed every `index.refresh_interval` which defaults to one second. That setting is [dynamic](#). Calling the [Refresh](#) API or setting `refresh` to `true` on any of the APIs that support it will also cause a refresh, in turn causing already running requests with `refresh=wait_for` to return.

`false` (the default)

Take no refresh related actions. The changes made by this request will be made visible at some point after the request returns.

Choosing which setting to use



Unless you have a good reason to wait for the change to become visible, always use `refresh=false` (the default setting). The simplest and fastest choice is to omit the `refresh` parameter from the URL.

If you absolutely must have the changes made by a request visible synchronously with the request, you must choose between putting more load on Elasticsearch (`true`) and waiting longer for the response (`wait_for`). Here are a few points that should inform that decision:

- The more changes being made to the index the more work `wait_for` saves compared to `true`. In the case that the index is only changed once every `index.refresh_interval` then it saves no work.
- `true` creates less efficient indexes constructs (tiny segments) that must later be merged into more efficient index constructs (larger segments). Meaning that the cost of `true` is paid at index time to create the tiny segment, at search time to search the tiny segment, and at merge time to make the larger segments.
- Never start multiple `refresh=wait_for` requests in a row. Instead batch them into a single bulk request with `refresh=wait_for` and Elasticsearch will start them all in parallel and return only when they have all finished.

Set up a cluster for high availability>

Snapshot and restore>

Secure the Elastic Stack>

Watcher>

Command line tools>

How to>

Troubleshooting>

REST APIs

API conventions

Common options

REST API compatibility

Autoscaling APIs>

Behavioral Analytics APIs>

Compact and aligned text (CAT) APIs>

Cluster APIs>

Cross-cluster replication APIs>

Data stream APIs>

Document APIs

Reading and Writing documents

Index

Get

Delete

Delete by query

Update

Update by query

Multi get

Bulk

Reindex

Term vectors

Multi term vectors

?refresh

Optimistic concurrency control

Enrich APIs>

EQL APIs>

Features APIs>

Fleet APIs>

Find structure API

Graph explore API

Index APIs>

Index lifecycle management APIs>

- If the refresh interval is set to `-1`, disabling the automatic refreshes, then requests with `refresh=wait_for` will wait indefinitely until some action causes a refresh. Conversely, setting `index.refresh_interval` to something shorter than the default like `200ms` will make `refresh=wait_for` come back faster, but it'll still generate inefficient segments.
- `refresh=wait_for` only affects the request that it is on, but, by forcing a refresh immediately, `refresh=true` will affect other ongoing request. In general, if you have a running system you don't wish to disturb then `refresh=wait_for` is a smaller modification.

`refresh=wait_for` Can Force a Refresh

If a `refresh=wait_for` request comes in when there are already `index.max_refresh_listeners` (defaults to 1000) requests waiting for a refresh on that shard then that request will behave just as though it had `refresh` set to `true` instead: it will force a refresh. This keeps the promise that when a `refresh=wait_for` request returns that its changes are visible for search while preventing unchecked resource usage for blocked requests. If a request forced a refresh because it ran out of listener slots then its response will contain `"forced_refresh": true`.

Bulk requests only take up one slot on each shard that they touch no matter how many times they modify the shard.

Examples

These will create a document and immediately refresh the index so it is visible:

```
PUT /test/_doc/1?refresh
{"test": "test"}
PUT /test/_doc/2?refresh=true
{"test": "test"}
```

Console

[Copy as curl](#) [View in Console](#)

These will create a document without doing anything to make it visible for search:

```
PUT /test/_doc/3
{"test": "test"}
PUT /test/_doc/4?refresh=false
{"test": "test"}
```

Console

[Copy as curl](#) [View in Console](#)

This will create a document and wait for it to become visible for search:

```
PUT /test/_doc/4?refresh=wait_for
{"test": "test"}
```

Console

[Copy as curl](#) [View in Console](#)

[« Multi term vectors API](#)

[Optimistic concurrency control »](#)

https://www.elastic.co/guide/en/elasticsearch/reference/current/docs-refresh.html

2/4